



SIPGN

Choosing a Vector Database

A practical guide to the tradeoffs between pgvector, Pinecone, and Qdrant for RAG

Project Engineer Team

2026-06-09 v1.0



About this document

COMPANY

SIPGN

AUTHOR

Project Engineer Team

EMAIL

harry.sitohang@inadigital.co.id

COPYRIGHT

© 2026

CONFIDENTIALITY

Confidential

Contents

- 1 Why This Guide Exists 1**
 - 1.1 How a RAG App Talks to the Vector Store 1
- 2 The Three Contenders 3**
 - 2.1 pgvector — your database, with vectors bolted on 3
 - 2.2 Pinecone — fully managed, hands-off scale 3
 - 2.3 Qdrant — open-source, purpose-built, self-or-cloud 4
- 3 Head-to-Head Comparison 5**
- 4 Recommendation 7**
 - 4.1 Glossary 7

Why This Guide Exists

Retrieval-augmented generation (RAG) lives or dies by its retrieval layer. The vector database is where your document embeddings sit and where every user query is matched against them. Pick the wrong one and you inherit operational pain, surprise bills, or a recall ceiling you can't engineer your way out of. Pick well and the store quietly disappears into the background of a fast, grounded app.

This handbook compares three popular choices — **pgvector**, **Pinecone**, and **Qdrant** — across the dimensions that actually move the needle: operational model, scale ceiling, filtering, cost, and how hard it is to get started. It is deliberately short. The goal is to get you to a defensible default for *your* situation, not to crown a universal winner.

In Plain English

A **vector database** stores numeric “fingerprints” (embeddings) of your text so the app can find passages that *mean* something similar to a question — not just passages that share the same keywords. RAG is the pattern of retrieving those passages and handing them to a language model so its answer is grounded in your data instead of its memory.

1.1 How a RAG App Talks to the Vector Store

Before comparing stores, it helps to see where the database sits in the request path. At query time the app turns the user's question into a



vector, asks the store for the nearest stored chunks, and feeds those chunks to the model as context. Documents are embedded and upserted ahead of time in an offline ingestion job.

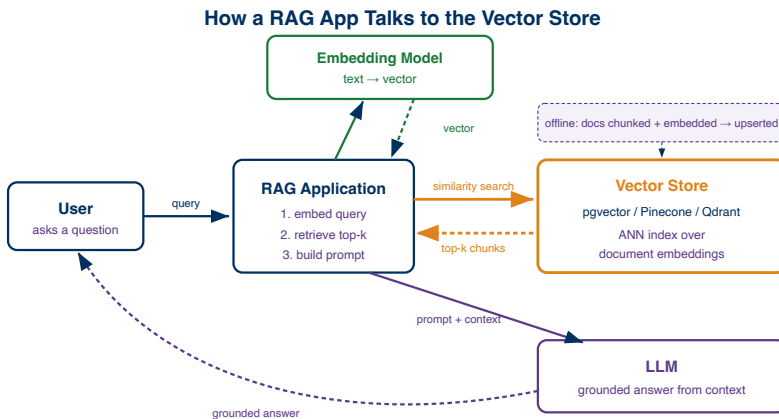


Figure 1.1: How a RAG app talks to the vector store: the app embeds the query, runs a similarity search against the vector store, retrieves the top-k chunks, and passes them with the prompt to the LLM.

The store's only job in this loop is **approximate nearest-neighbour (ANN) search**: given a query vector, return the k most similar document vectors fast, ideally while honouring metadata filters (tenant, language, recency). Everything that distinguishes the three contenders is some variation on *how well*, *how cheaply*, and *how operably* they do exactly that.

The Three Contenders

2.1 pgvector — your database, with vectors bolted on

[pgvector](#) is a PostgreSQL extension that adds a **vector** column type plus HNSW and IVFFlat indexes. If you already run Postgres, you add vectors without adding a new system: same backups, same transactions, same SQL, same access control. Joins between vectors and your relational data are trivial because they live in one place.

The tradeoff is that Postgres was not born to be a vector engine. At tens of millions of vectors with heavy concurrent search, you will be tuning HNSW parameters, watching memory, and eventually scaling Postgres itself — work that a purpose-built store handles for you.

2.2 Pinecone — fully managed, hands-off scale

[Pinecone](#) is a managed vector database: there is no server to run, no index to babysit. You upsert vectors and query them through an API, and it scales horizontally behind the scenes. For teams that want to ship retrieval without owning infrastructure, that convenience is the whole pitch.

You pay for it in two ways: a recurring bill that grows with stored vectors and query volume, and the constraints of a closed, hosted system — your embeddings live with a third party and you tune within the knobs they expose.



2.3 Qdrant — open-source, purpose-built, self-or-cloud

[Qdrant](#) is an open-source vector database written in Rust, with strong metadata filtering, payload indexing, and quantization for memory savings. You can self-host it (Docker, Kubernetes) or use Qdrant Cloud, so you keep the option to move. It sits between the other two: more capable and scalable than pgvector for pure vector work, more controllable and portable than Pinecone.

The cost is operational ownership when self-hosting — you run, monitor, and upgrade it — and a smaller (though fast-growing) ecosystem than Postgres.

Pro Tip

Start with **pgvector** if you already run **Postgres** and your corpus is under a few million vectors — you avoid a new system entirely and can always migrate later. Reach for a dedicated store **only when** recall, latency at scale, or filtering complexity actually hurt. Premature adoption of a specialized database is a common, avoidable source of cost and operational drag.



Head-to-Head Comparison

The contenders differ less on raw search quality (all three do competent ANN) and more on *operational shape*: who runs it, how it scales, and what it costs.

Dimension	pgvector	Pinecone	Qdrant
Model	Postgres extension	Fully managed SaaS	Open-source; self-host or cloud
Operational burden	Low if you run Postgres already	Lowest — nothing to operate	Medium self-hosted; low on cloud
Scale ceiling	Millions (tuning needed higher)	Very high, managed automatically	High; horizontal + quantization
Metadata filtering	Full SQL WHERE	Metadata filters via API	Rich payload filtering, indexed
Cost model	Your existing Postgres cost	Recurring usage-based bill	Compute you run, or cloud tier
Data control	Your own database	Third-party hosted	Yours (self-host) or managed cloud
Best when	Already on Postgres, modest scale	Want zero ops, predictable API	Want power + portability + filters

Note

There is no single “best” store — the right answer is the one whose *operational shape* matches your team. A two-person startup already on Postgres and a platform team running Kubernetes at scale will rationally pick differently from the very same feature list.



Recommendation

If you already run Postgres and your corpus is modest, **pgvector** is the lowest-friction starting point and is trivial to migrate away from later. If you want to ship retrieval with zero infrastructure ownership and a predictable API, **Pinecone** removes operational work at a recurring cost. If you need strong filtering, larger scale than pgvector handles comfortably, and the freedom to self-host or move, **Qdrant** is the most flexible. Decide on operational fit first; raw search quality is rarely the deciding factor.

4.1 Glossary

Term	Meaning
RAG	Retrieval-augmented generation — retrieve relevant text, then ground an LLM’s answer in it.
Embedding	A numeric vector representation of text capturing its meaning.
Vector database	A store optimized for similarity search over embedding vectors.
ANN	Approximate nearest-neighbour — fast similarity search that trades a little exactness for speed.
HNSW	Hierarchical Navigable Small World — a graph-based ANN index used by pgvector and Qdrant.

Term	Meaning
IVFFlat	An inverted-file ANN index in pgvector that clusters vectors into lists.
top-k	The k most similar chunks returned for a query.
Upsert	Insert or update a vector (and its metadata) in the store.
Payload / metadata	Non-vector fields stored beside a vector and used to filter results.
Quantization	Compressing vectors to reduce memory at a small accuracy cost.